

## Login + Registration (Servlet + JSP + JDBC)

Below is the **complete Login + Registration project (Servlet + JSP + JDBC)** with full folder structure, SQL table, JSP pages, Servlet code, and DAO layer.

---

### ✓ Project: Login + Registration System (Servlet + JSP + JDBC)

**Tech Stack:** JSP, Servlet, JDBC, MySQL

**Pattern Used:** MVC

---

## 1. Database Setup

### Create Database & Table

sql

 Copy code

```
CREATE DATABASE userdb;

USE userdb;

CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    email VARCHAR(100) UNIQUE,
    password VARCHAR(100)
);
```

---

## 2. Project Structure

pgsql

 Copy code

```
WebContent/
|— index.jsp
```

```
|— login.jsp
|— register.jsp
|— home.jsp
└─ WEB-INF/
    └─ web.xml
```

```
src/
└─ com.yuvraj.servlet/
    ├── RegisterServlet.java
    └── LoginServlet.java
└─ com.yuvraj.dao/
    └── UserDao.java
└─ com.yuvraj.db/
    └── DBConnect.java
```

---

### 3. DBConnect.java

java

 Copy code

```
package com.yuvraj.db;

import java.sql.Connection;
import java.sql.DriverManager;

public class DBConnect {
    private static Connection conn;

    public static Connection getConn() {
        try {
            if (conn == null) {
                Class.forName("com.mysql.cj.jdbc.Driver");
                conn = DriverManager.getConnection(
                    "jdbc:mysql://localhost:3306/userdb",
                    "root",
                    "your_password"
                );
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return conn;
    }
}
```

```
}  
}
```

---

## 4. UserDao.java

java

 Copy code

```
package com.yuvraj.dao;  
  
import java.sql.*;  
import com.yuvraj.db.DBConnect;  
import com.yuvraj.model.User;  
  
public class UserDao {  
  
    public boolean userRegister(User user) {  
        boolean f = false;  
  
        try {  
            Connection conn = DBConnect.getConn();  
            String sql = "INSERT INTO users(name,email,password) VALUES (?  
            PreparedStatement ps = conn.prepareStatement(sql);  
  
            ps.setString(1, user.getName());  
            ps.setString(2, user.getEmail());  
            ps.setString(3, user.getPassword());  
  
            int i = ps.executeUpdate();  
            if (i == 1) f = true;  
  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
        return f;  
    }  
  
    public User login(String email, String password) {  
        User u = null;  
        try {  
            Connection conn = DBConnect.getConn();  
            String sql = "SELECT * FROM users WHERE email=? AND password=?"
```

```
        PreparedStatement ps = conn.prepareStatement(sql);

        ps.setString(1, email);
        ps.setString(2, password);

        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            u = new User();
            u.setId(rs.getInt("id"));
            u.setName(rs.getString("name"));
            u.setEmail(rs.getString("email"));
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
    return u;
}
}
```

---

## 5. Model Class User.java

java

 Copy code

```
package com.yuvraj.model;

public class User {
    private int id;
    private String name;
    private String email;
    private String password;

    // getters and setters
}
```

---

## 6. RegisterServlet.java

java

 Copy code

```

package com.yuvraj.servlet;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;

import com.yuvraj.dao.UserDao;
import com.yuvraj.model.User;

public class RegisterServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String name = req.getParameter("name");
        String email = req.getParameter("email");
        String password = req.getParameter("password");

        User user = new User();
        user.setName(name);
        user.setEmail(email);
        user.setPassword(password);

        UserDao dao = new UserDao();
        boolean f = dao.userRegister(user);

        if (f) {
            resp.sendRedirect("login.jsp?msg=success");
        } else {
            resp.sendRedirect("register.jsp?msg=failed");
        }
    }
}

```

---

## 7. LoginServlet.java

java

 Copy code

```

package com.yuvraj.servlet;

```

```

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;

import com.yuvraj.dao.UserDao;
import com.yuvraj.model.User;

public class LoginServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String email = req.getParameter("email");
        String password = req.getParameter("password");

        UserDao dao = new UserDao();
        User user = dao.login(email, password);

        if (user != null) {
            HttpSession session = req.getSession();
            session.setAttribute("user", user);
            resp.sendRedirect("home.jsp");
        } else {
            resp.sendRedirect("login.jsp?msg=invalid");
        }
    }
}

```

---

## 8. web.xml Configuration

xml

 Copy code

```

<web-app>

    <servlet>
        <servlet-name>register</servlet-name>
        <servlet-class>com.yuvraj.servlet.RegisterServlet</servlet-class>
    </servlet>

    <servlet-mapping>

```

```
<servlet-name>register</servlet-name>
<url-pattern>/register</url-pattern>
</servlet-mapping>

<servlet>
  <servlet-name>login</servlet-name>
  <servlet-class>com.yuvraj.servlet.LoginServlet</servlet-class>
</servlet>

<servlet-mapping>
  <servlet-name>login</servlet-name>
  <url-pattern>/login</url-pattern>
</servlet-mapping>

</web-app>
```

---

## 9. JSP Pages

---

### register.jsp

jsp

 Copy code

```
<form action="register" method="post">
  <h2>Registration</h2>
  Name: <input type="text" name="name"><br><br>
  Email: <input type="email" name="email"><br><br>
  Password: <input type="password" name="password"><br><br>
  <button type="submit">Register</button>
</form>
```

---

### login.jsp

jsp

 Copy code

```
<form action="login" method="post">
  <h2>Login</h2>
  Email: <input type="email" name="email"><br><br>
  Password: <input type="password" name="password"><br><br>
```

```
<button type="submit">Login</button>
</form>
```

---

## 👉 home.jsp

jsp

 Copy code

```
<%
    com.yuvraj.model.User user = (com.yuvraj.model.User) session.getAttribute(
        "user");
    if (user == null) {
        response.sendRedirect("login.jsp");
    }
%>

<h2>Welcome, <%= user.getName() %>!</h2>
<a href="logout.jsp">Logout</a>
```

---

## 👉 logout.jsp

jsp

 Copy code

```
<%
    session.invalidate();
    response.sendRedirect("login.jsp");
%>
```

---

## 🎉 Your Login + Registration Project Is Ready!

If you want, I can also give you:

- ✅ WAR file
- ✅ CSS UI Design
- ✅ Session filter
- ✅ Forgot Password Module
- ✅ Role-based Login
- ✅ Admin + User Dashboard



Just tell me **Next**.

## Login + Register with Password Encryption (SHA-256)

Below is the **complete Login + Registration (Servlet + JSP + JDBC) with SHA-256 Password Encryption**.

This is production-recommended and secure.

---

### Why SHA-256?

Because passwords should **never be stored in plain text**.

SHA-256 converts the password into a **64-character irreversible hash**.

---

### 1. Password Hashing Utility (SHA-256)

Create a helper class:

PasswordEncrypt.java

java

 Copy code

```
package com.yuvraj.security;

import java.security.MessageDigest;

public class PasswordEncrypt {

    public static String hashPassword(String password) {
        StringBuilder sb = new StringBuilder();
        try {
            MessageDigest md = MessageDigest.getInstance("SHA-256");
            byte[] bytes = md.digest(password.getBytes());

            for (byte b : bytes) {
                sb.append(String.format("%02x", b));
            }
        }
    }
}
```

```
        } catch (Exception e) {
            e.printStackTrace();
        }
        return sb.toString();
    }
}
```

- ✓ Converts normal password → **SHA-256 encrypted hash**
  - ✓ Safe for database storage
- 

## ✓ 2. Update User Table

Your database remains same:

sql

 Copy code

```
CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    email VARCHAR(100) UNIQUE,
    password VARCHAR(100)
);
```

Password is stored as hashed:

 Copy code

e3afed0047b08059d0fada10f400c1e5d...

---

## ✓ 3. Modify Registration Servlet (Encrypt Password)

RegisterServlet.java

java

 Copy code

```
package com.yuvraj.servlet;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;
```

```

import com.yuvraj.dao.UserDao;
import com.yuvraj.model.User;
import com.yuvraj.security.PasswordEncrypt;

public class RegisterServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String name = req.getParameter("name");
        String email = req.getParameter("email");
        String password = req.getParameter("password");

        // Encrypt password
        String encryptedPassword = PasswordEncrypt.hashPassword(password);

        User user = new User();
        user.setName(name);
        user.setEmail(email);
        user.setPassword(encryptedPassword);

        UserDao dao = new UserDao();
        boolean f = dao.userRegister(user);

        if (f) {
            resp.sendRedirect("login.jsp?msg=registered");
        } else {
            resp.sendRedirect("register.jsp?msg=failed");
        }
    }
}

```

---

## 4. Modify Login Servlet (Encrypt Before Comparing)

LoginServlet.java

java

 Copy code

```

package com.yuvraj.servlet;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;

import com.yuvraj.dao.UserDao;
import com.yuvraj.model.User;
import com.yuvraj.security.PasswordEncrypt;

public class LoginServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String email = req.getParameter("email");
        String password = req.getParameter("password");

        // Encrypt entered password
        String encryptedPassword = PasswordEncrypt.hashPassword(password);

        UserDao dao = new UserDao();
        User user = dao.login(email, encryptedPassword);

        if (user != null) {
            HttpSession session = req.getSession();
            session.setAttribute("user", user);
            resp.sendRedirect("home.jsp");
        } else {
            resp.sendRedirect("login.jsp?msg=invalid");
        }
    }
}

```

---

## 5. Update DAO (No Change Needed Except Query)

UserDao.java

java

 Copy code

```
public User login(String email, String encryptedPassword) {
    User u = null;
    try {
        Connection conn = DBConnect.getConn();
        String sql = "SELECT * FROM users WHERE email=? AND password=?";
        PreparedStatement ps = conn.prepareStatement(sql);

        ps.setString(1, email);
        ps.setString(2, encryptedPassword);

        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            u = new User();
            u.setId(rs.getInt("id"));
            u.setName(rs.getString("name"));
            u.setEmail(rs.getString("email"));
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
    return u;
}
```

---

## 6. JSP Pages (Same as Before)

No changes needed in JSP pages (login.jsp & register.jsp).

---

### FINAL OUTPUT:

- ✓ Your system will store encrypted passwords
  - ✓ No user can see original password
  - ✓ Even DB admin cannot decode password
- 

### Want to Add More?

I can also add:

- ✓ SHA-256 with **Salt + Pepper** (more secure)
- ✓ Forgot Password with OTP
- ✓ Email Verification
- ✓ Role-based login (Admin/User)
- ✓ Bootstrap UI for Login & Register

Just say **Next**.

## Login + Session Management

Below is a clear and complete **Login + Session Management** implementation using **Servlet + JSP + JDBC**.

This includes:

- ✓ Secure login
- ✓ Session creation
- ✓ Session validation
- ✓ Session timeout
- ✓ Logout
- ✓ Session filter (recommended)

---

### ✓ 1. Login Flow (How Session Works)

When user logs in:

- Server verifies email + password
- If correct → create session:

java

 Copy code

```
HttpSession session = request.getSession();  
session.setAttribute("userObj", user);
```

- Redirect to **home.jsp**

When opening protected pages:

- System checks → Is session active?
- If NO session → redirect to login.jsp

### When logout:

- Session destroyed:

java

 Copy code

```
session.invalidate();
```

---

## 2. LoginServlet.java

(Session Created after successful login)

java

 Copy code

```
package com.yuvraj.servlet;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;

import com.yuvraj.dao.UserDao;
import com.yuvraj.model.User;
import com.yuvraj.security.PasswordEncrypt;

public class LoginServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String email = req.getParameter("email");
        String password = req.getParameter("password");

        // Hash password before comparing
        String hashedPass = PasswordEncrypt.hashPassword(password);

        UserDao dao = new UserDao();
        User user = dao.login(email, hashedPass);

        if (user != null) {
```

```

        HttpSession session = req.getSession();
        session.setAttribute("userObj", user);

        // Optional: Set session timeout (minutes)
        session.setMaxInactiveInterval(10 * 60); // 10 minutes

        resp.sendRedirect("home.jsp");

    } else {
        resp.sendRedirect("login.jsp?msg=invalid");
    }
}
}

```

---

### ✓ 3. Session Check Filter (Recommended)

This filter blocks access to any JSP/Servlet without login.

#### AuthFilter.java

java

 Copy code

```

package com.yuvraj.filter;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;

public class AuthFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
        throws IOException, ServletException {

        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse resp = (HttpServletResponse) response;

        HttpSession session = req.getSession(false);

        boolean loggedIn = (session != null && session.getAttribute("userObj") != null);
    }
}

```



```

// Pages that don't require login
String path = req.getRequestURI();

if (loggedIn || path.endsWith("login.jsp") || path.endsWith("register.jsp") ||
    path.endsWith("LoginServlet") || path.endsWith("RegisterServlet")) {

    chain.doFilter(request, response);
} else {
    resp.sendRedirect("login.jsp?msg=sessionExpired");
}
}
}

```

## ✓ 4. Add Filter in web.xml

xml

 Copy code

```

<filter>
    <filter-name>AuthFilter</filter-name>
    <filter-class>com.yuvraj.filter.AuthFilter</filter-class>
</filter>

<filter-mapping>
    <filter-name>AuthFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>

```

- ✓ Protects all pages except login/register
- ✓ User cannot open URL directly
- ✓ Most important part of session security

## ✓ 5. home.jsp (Session Validation)

You must block users who come without login.

jsp

 Copy code

```

<%
    com.yuvraj.model.User user = (com.yuvraj.model.User) session.getAttribute(

```

```
        if (user == null) {
            response.sendRedirect("login.jsp");
            return;
        }
    %>

    <h2>Welcome, <%= user.getName() %></h2>

    <a href="logout.jsp">Logout</a>
```

---

## ✓ 6. logout.jsp (Destroy Session)

jsp

 Copy code

```
<%
    session.invalidate();
    response.sendRedirect("login.jsp?msg=logoutSuccess");
%>
```

- ✓ Clears session
  - ✓ Redirects to login page
- 

## ✓ 7. login.jsp (Show Message)

jsp

 Copy code

```
<% if (request.getParameter("msg") != null) { %>
    <p style="color:red;"><%= request.getParameter("msg") %></p>
<% } %>

<form action="LoginServlet" method="post">
    <h2>Login</h2>
    Email: <input type="email" name="email">
    Password: <input type="password" name="password">
    <button type="submit">Login</button>
</form>
```

---

# Complete Session Management Features Provided

- ✓ Secure login
  - ✓ SHA-256 encrypted password
  - ✓ Session create
  - ✓ Session timeout (10 min)
  - ✓ Session filter
  - ✓ Prevent direct URL access
  - ✓ Logout
- 

## ★ Want me to add more features?

I can add:

- ✓ "Remember Me" using Cookies
- ✓ Auto-login token
- ✓ Session tracking in MySQL
- ✓ Role-based security (Admin/User)
- ✓ JWT Login (Most modern)

Just say **Next** or tell me what you want.

Login + Forgot Password (OTP email)

Below is a **complete, production-style "Login + Forgot Password (OTP Email)"** system using **Servlet + JSP + JDBC**.

This includes:

- ✓ Login (with SHA-256 password encryption)
- ✓ Forgot Password Screen
- ✓ Send OTP to Email
- ✓ OTP Validation
- ✓ Reset Password
- ✓ Update in Database
- ✓ Secure Best Practices

---

## Flow Diagram

pgsql

 Copy code

```
Forgot Password →  
Enter Email →  
Generate OTP →  
Send OTP email →  
Verify OTP →  
Reset New Password →  
Encrypt (SHA-256) →  
Update DB →  
Login Again
```

---

## ★ 1. Add Email Dependency (JavaMail API)

Add jars in WEB-INF/lib :

- javax.mail.jar
- activation.jar

---

## ★ 2. Database Table (users)

(No change needed)

We only add OTP temporarily in session.

---

## ★ 3. Forgot Password Page

forgot\_password.jsp

jsp

 Copy code

```
<h2>Forgot Password</h2>  
  
<form action="ForgotPasswordServlet" method="post">  
    Enter Registered Email: <br>  
    <input type="email" name="email" required>
```

```

        <button type="submit">Send OTP</button>
    </form>

    <% if(request.getParameter("msg") != null){ %>
        <p style="color:red;"><%= request.getParameter("msg") %></p>
    <% } %>

```

---

## ★ 4. OTP Email Sender Utility

### EmailUtil.java

java

 Copy code

```

package com.yuvraj.util;

import java.util.Properties;
import javax.mail.*;
import javax.mail.internet.*;

public class EmailUtil {

    public static void sendEmail(String toEmail, String subject, String message) {

        String fromEmail = "yourEmail@gmail.com";
        String password = "your-app-password"; // use App Password

        Properties props = new Properties();
        props.put("mail.smtp.host", "smtp.gmail.com");
        props.put("mail.smtp.port", "587");
        props.put("mail.smtp.auth", "true");
        props.put("mail.smtp.starttls.enable", "true");

        Session session = Session.getInstance(props,
            new javax.mail.Authenticator() {
                protected PasswordAuthentication getPasswordAuthentication() {
                    return new PasswordAuthentication(fromEmail, password);
                }
            });

        try {
            Message messageObj = new MimeMessage(session);

```

```

        message.setFrom(new InternetAddress(fromEmail));
        message.setRecipients(Message.RecipientType.TO, InternetAddresss);
        message.setSubject(subject);
        message.setText(messageText);
        Transport.send(message);

    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

---

## ★ 5. ForgotPasswordServlet.java

Generates OTP → sends email → stores OTP in session.

### ForgotPasswordServlet.java

java

 Copy code

```

package com.yuvraj.servlet;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;
import java.util.Random;

import com.yuvraj.dao.UserDao;
import com.yuvraj.model.User;
import com.yuvraj.util.EmailUtil;

public class ForgotPasswordServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String email = req.getParameter("email");

        UserDao dao = new UserDao();
        User user = dao.getUserByEmail(email);
    }
}

```

```

        if (user == null) {
            resp.sendRedirect("forgot_password.jsp?msg=Email Not Found");
            return;
        }

        // Generate 6-digit OTP
        int otp = new Random().nextInt(900000) + 100000;

        // Store OTP in session
        HttpSession session = req.getSession();
        session.setAttribute("otp", otp);
        session.setAttribute("email", email);

        // Send OTP Email
        String msg = "Your OTP for password reset is: " + otp;
        EmailUtil.sendEmail(email, "Password Reset OTP", msg);

        resp.sendRedirect("verify_otp.jsp");
    }
}

```

---

## ★ 6. OTP Verification Page

### verify\_otp.jsp

jsp

 Copy code

```

<h2>Verify OTP</h2>

<form action="VerifyOtpServlet" method="post">
    Enter OTP: <br>
    <input type="number" name="otp" required>
    <button type="submit">Verify</button>
</form>

<% if(request.getParameter("msg") != null){ %>
    <p style="color:red;"><%= request.getParameter("msg") %></p>
<% } %>

```

---

## ★ 7. VerifyOtpServlet.java

### VerifyOtpServlet.java

java

 Copy code

```
package com.yuvraj.servlet;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;

public class VerifyOtpServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        int enteredOtp = Integer.parseInt(req.getParameter("otp"));

        HttpSession session = req.getSession();
        int sessionOtp = (int) session.getAttribute("otp");

        if (enteredOtp == sessionOtp) {
            resp.sendRedirect("reset_password.jsp");
        } else {
            resp.sendRedirect("verify_otp.jsp?msg=Invalid OTP");
        }
    }
}
```

---

## ★ 8. Reset Password Page

### reset\_password.jsp

jsp

 Copy code

```
<h2>Reset Password</h2>

<form action="ResetPasswordServlet" method="post">
    New Password: <br>
    <input type="password" name="password" required>
```



```
<button type="submit">Reset Password</button>
</form>
```

---

## ★ 9. ResetPasswordServlet.java

### ResetPasswordServlet.java

java

 Copy code

```
package com.yuvraj.servlet;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;

import com.yuvraj.dao.UserDao;
import com.yuvraj.security.PasswordEncrypt;

public class ResetPasswordServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        HttpSession session = req.getSession();
        String email = (String) session.getAttribute("email");

        String newPass = req.getParameter("password");
        String encryptedPass = PasswordEncrypt.hashPassword(newPass);

        UserDao dao = new UserDao();
        boolean f = dao.updatePassword(email, encryptedPass);

        if (f) {
            session.invalidate();
            resp.sendRedirect("login.jsp?msg=Password Updated");
        } else {
            resp.sendRedirect("reset_password.jsp?msg=Failed");
        }
    }
}
```

---

## ★ 10. DAO Method (update password)

Add in UserDao.java

java

 Copy code

```
public boolean updatePassword(String email, String pass) {
    boolean f = false;
    try {
        Connection conn = DBConnect.getConn();
        PreparedStatement ps = conn.prepareStatement(
            "UPDATE users SET password=? WHERE email=?"
        );
        ps.setString(1, pass);
        ps.setString(2, email);

        int i = ps.executeUpdate();
        if (i == 1) f = true;

    } catch (Exception e) {
        e.printStackTrace();
    }
    return f;
}
```

---

## Your Forgot Password System is Ready!

- ✓ Login
- ✓ Forgot Password
- ✓ OTP Email verification
- ✓ Reset Password
- ✓ Password encrypted using SHA-256
- ✓ Secure session handling

---

## Want Next Feature?

I can add:

- ✓ Change Password (inside profile)
- ✓ Resend OTP
- ✓ OTP expiry system
- ✓ Remember Me (Cookies)
- ✓ Block user after 3 wrong OTP attempts
- ✓ Email templates (HTML)

Just say **Next!**

Next

Nice — I implemented the **Next** features you asked for and gave you a complete, ready-to-drop-into-your project set of changes. I added:

- Change Password (profile)
- Resend OTP (with limits)
- OTP expiry (configurable)
- Block user after 3 wrong OTP attempts (with unlock time)
- HTML email templates for OTP (EmailUtil update)

I'll give SQL changes, DAO methods, servlets, JSP snippets, and brief wiring notes you can integrate into your existing project. I made reasonable defaults (5-minute OTP expiry, 3 wrong attempts → lock for 15 minutes, resend limit 3 per hour) — change the constants as you like.

---

## 1) Database changes

Run these SQL statements to add OTP storage and lock fields:

sql

 Copy code

```
-- OTP table
CREATE TABLE password_reset_otp (
  id INT AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(255) NOT NULL,
  otp VARCHAR(10) NOT NULL,
  created_at DATETIME NOT NULL,
  attempts INT DEFAULT 0,
```

```
    sent_count INT DEFAULT 1,          -- how many OTPs sent in current window
    last_sent_at DATETIME,
    UNIQUE(email)
);

-- Add lock fields to users table
ALTER TABLE users
    ADD COLUMN is_locked TINYINT(1) DEFAULT 0,
    ADD COLUMN lock_until DATETIME NULL;
```

#### Notes:

- password\_reset\_otp keeps one row per email (unique). We update it on resend and verify.
  - sent\_count + last\_sent\_at helps enforce resend limits.
- 

## 2) Constants (config) — put in a central class

Create a small config class:

java

 Copy code

```
package com.yuvraj.config;

import java.time.Duration;

public class SecurityConfig {
    public static final int OTP_EXPIRY_MINUTES = 5;
    public static final int OTP_MAX_WRONG_ATTEMPTS = 3;
    public static final int LOCK_MINUTES_AFTER_FAILED_OTP = 15;
    public static final int RESEND_LIMIT_PER_HOUR = 3;
}
```

---

## 3) DAO additions ( UserDao.java and OtpDao.java )

Add methods to manage OTPs and locking. I suggest creating an OtpDao for clarity.

OtpDao.java

java

 Copy code

```

package com.yuvraj.dao;

import java.sql.*;
import java.time.LocalDateTime;
import com.yuvraj.db.DBConnect;

public class OtpDao {

    // Insert or update an OTP row for email
    public boolean upsertOtp(String email, String otp) {
        try {
            Connection conn = DBConnect.getConn();
            String sqlSelect = "SELECT id FROM password_reset_otp WHERE email=?";
            PreparedStatement psSel = conn.prepareStatement(sqlSelect);
            psSel.setString(1, email);
            ResultSet rs = psSel.executeQuery();

            LocalDateTime now = LocalDateTime.now();
            if (rs.next()) {
                String sqlUpdate = "UPDATE password_reset_otp SET otp=?, createdAt=?, updatedAt=? WHERE id=?";
                PreparedStatement psUpd = conn.prepareStatement(sqlUpdate);
                psUpd.setString(1, otp);
                psUpd.setTimestamp(2, Timestamp.valueOf(now));
                psUpd.setTimestamp(3, Timestamp.valueOf(now));
                psUpd.setString(4, email);
                psUpd.executeUpdate();
            } else {
                String sqlInsert = "INSERT INTO password_reset_otp(email, otp, createdAt, updatedAt, id, isVerified) VALUES(?, ?, ?, ?, ?, ?)";
                PreparedStatement psIns = conn.prepareStatement(sqlInsert);
                psIns.setString(1, email);
                psIns.setString(2, otp);
                psIns.setTimestamp(3, Timestamp.valueOf(now));
                psIns.setInt(4, 0);
                psIns.setInt(5, 1);
                psIns.setTimestamp(6, Timestamp.valueOf(now));
                psIns.executeUpdate();
            }
            return true;
        } catch (Exception e) {
            e.printStackTrace();
            return false;
        }
    }
}

```

```

    }
}

// Get OTP row (email)
public ResultSet getOtpRow(String email) {
    try {
        Connection conn = DBConnect.getConn();
        PreparedStatement ps = conn.prepareStatement("SELECT * FROM password_reset_otp WHERE email = ?");
        ps.setString(1, email);
        return ps.executeQuery();
    } catch (Exception e) {
        e.printStackTrace();
        return null;
    }
}

// Increase attempts and return updated attempts
public int incrementAttempts(String email) {
    int attempts = -1;
    try {
        Connection conn = DBConnect.getConn();
        PreparedStatement ps = conn.prepareStatement(
            "UPDATE password_reset_otp SET attempts = attempts + 1 WHERE email = ?");
        ps.setString(1, email);
        ps.executeUpdate();

        PreparedStatement ps2 = conn.prepareStatement("SELECT attempts FROM password_reset_otp WHERE email = ?");
        ps2.setString(1, email);
        ResultSet rs = ps2.executeQuery();
        if (rs.next()) attempts = rs.getInt("attempts");
    } catch (Exception e) { e.printStackTrace(); }
    return attempts;
}

// Reset OTP record (after successful reset)
public boolean deleteOtp(String email) {
    try {
        Connection conn = DBConnect.getConn();
        PreparedStatement ps = conn.prepareStatement("DELETE FROM password_reset_otp WHERE email = ?");
        ps.setString(1, email);
        ps.executeUpdate();
        return true;
    }
}

```

```

    } catch (Exception e) { e.printStackTrace(); return false; }
}

// Check resend limit: determine sent_count within last hour
public int getSentCountSince(String email) {
    try {
        Connection conn = DBConnect.getConn();
        PreparedStatement ps = conn.prepareStatement("SELECT sent_count1
ps.setString(1, email);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            Timestamp lastSent = rs.getTimestamp("last_sent_at");
            int sentCount = rs.getInt("sent_count");
            if (lastSent == null) return 0;
            // Caller will check time window
            return sentCount;
        }
    } catch (Exception e) { e.printStackTrace(); }
    return 0;
}

// Reset sent_count (if last_sent_at older than window)
public void resetSentCount(String email) {
    try {
        Connection conn = DBConnect.getConn();
        PreparedStatement ps = conn.prepareStatement("UPDATE password_1
ps.setString(1, email);
        ps.executeUpdate();
    } catch (Exception e) { e.printStackTrace(); }
}

}

```

## UserDao.java additions

java

 Copy code

```

// Check if user is locked
public boolean isUserLocked(String email) {
    try {
        Connection conn = DBConnect.getConn();
        PreparedStatement ps = conn.prepareStatement("SELECT is_locked, loc
ps.setString(1, email);

```

```

ResultSet rs = ps.executeQuery();
if (rs.next()) {
    boolean locked = rs.getInt("is_locked") == 1;
    Timestamp lockUntil = rs.getTimestamp("lock_until");
    if (locked && lockUntil != null) {
        if (lockUntil.before(new Timestamp(System.currentTimeMillis()
            // unlock automatically
            PreparedStatement psu = conn.prepareStatement("UPDATE users SET is_locked=0 WHERE email=?");
            psu.setString(1, email);
            psu.executeUpdate();
            return false;
        }
        return true;
    }
}
} catch (Exception e) { e.printStackTrace(); }
return false;
}

// Lock user until a future timestamp
public boolean lockUser(String email, java.sql.Timestamp until) {
    try {
        Connection conn = DBConnect.getConn();
        PreparedStatement ps = conn.prepareStatement("UPDATE users SET is_locked=1 WHERE email=?");
        ps.setTimestamp(1, until);
        ps.setString(2, email);
        ps.executeUpdate();
        return true;
    } catch (Exception e) { e.printStackTrace(); return false; }
}

```

---

## 4) Email utility: HTML template & resend-friendly

Update your `EmailUtil` to send HTML emails and accept template content.

java

 Copy code

```

package com.yuvraj.util;

import java.util.Properties;
import javax.mail.*;
import javax.mail.internet.*;

```



```

public class EmailUtil {

    private static final String FROM_EMAIL = "yourEmail@gmail.com";
    private static final String FROM_PASSWORD = "your-app-password";

    public static void sendHtmlEmail(String toEmail, String subject, String
        Properties props = new Properties();
        props.put("mail.smtp.host", "smtp.gmail.com");
        props.put("mail.smtp.port", "587");
        props.put("mail.smtp.auth", "true");
        props.put("mail.smtp.starttls.enable", "true");

        Session session = Session.getInstance(props,
            new javax.mail.Authenticator() {
                protected PasswordAuthentication getPasswordAuthentication() {
                    return new PasswordAuthentication(FROM_EMAIL, FROM_
                }
            });

        try {
            Message message = new MimeMessage(session);
            message.setFrom(new InternetAddress(FROM_EMAIL, "Your App Name");
            message.setRecipients(Message.RecipientType.TO, InternetAddress
            message.setSubject(subject);
            message.setContent(htmlMessage, "text/html; charset=utf-8");
            Transport.send(message);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    // Helper to create HTML OTP template
    public static String otpTemplate(String otp, String email) {
        return "<html><body>"
            + "<h2>Password Reset OTP</h2>"
            + "<p>Hi,</p>"
            + "<p>Your OTP to reset the password for <b>" + email + "</b> :
            + "<h1 style='letter-spacing:4px;'>" + otp + "</h1>"
            + "<p>This OTP expires in " + com.yuvraj.config.SecurityConfig
            + "<p>If you didn't request this, please ignore this email.</p>
            + "<hr><small>Do not share this OTP with anyone.</small>"
            + "</body></html>";
    }
}

```

```
}
```

```
}
```

---

## 5) Servlets

I'll give three servlets: `ForgotPasswordServlet` (send/new OTP or resend), `ResendOtpServlet` (explicit resend), and `VerifyOtpServlet` (with attempt counting & lock logic). Also `ChangePasswordServlet` for logged-in users.

### `ForgotPasswordServlet` (send initial OTP)

java

 Copy code

```
package com.yuvraj.servlet;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;
import java.time.LocalDateTime;
import java.time.ZoneId;
import java.sql.Timestamp;
import java.util.Random;

import com.yuvraj.dao.UserDao;
import com.yuvraj.dao.OtpDao;
import com.yuvraj.util.EmailUtil;
import com.yuvraj.config.SecurityConfig;

public class ForgotPasswordServlet extends HttpServlet {
    private OtpDao otpDao = new OtpDao();
    private UserDao userDao = new UserDao();

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String email = req.getParameter("email");

        // check user exists
        if (userDao.getUserByEmail(email) == null) {
            resp.sendRedirect("forgot_password.jsp?msg=Email+not+found");
        }
    }
}
```

```

        return;
    }

    // check if user locked
    if (userDao.isUserLocked(email)) {
        resp.sendRedirect("forgot_password.jsp?msg=Account+locked.+Try-
        return;
    }

    // resend limit: if record exists and last_sent_at within 1 hour at
    try {
        java.sql.ResultSet rs = otpDao.getOtpRow(email);
        boolean allowSend = true;
        if (rs != null && rs.next()) {
            Timestamp lastSent = rs.getTimestamp("last_sent_at");
            int sentCount = rs.getInt("sent_count");
            if (lastSent != null) {
                long millisSince = System.currentTimeMillis() - lastSer
                long hourMillis = 60L * 60L * 1000L;
                if (millisSince <= hourMillis && sentCount >= Security(
                    allowSend = false;
                } else if (millisSince > hourMillis) {
                    otpDao.resetSentCount(email);
                }
            }
        }

        if (!allowSend) {
            resp.sendRedirect("forgot_password.jsp?msg=Resend+limit+re;
            return;
        }
    } catch (Exception e) { e.printStackTrace(); }

    // generate OTP
    int otp = new Random().nextInt(900000) + 100000;
    String otpStr = String.valueOf(otp);

    // upsert OTP row
    otpDao.upsertOtp(email, otpStr);

    // Send email
    String html = EmailUtil.otpTemplate(otpStr, email);

```

```

        EmailUtil.sendHtmlEmail(email, "Your Password Reset OTP", html);

        // store email in session to use in verify/reset pages (optional)
        HttpSession session = req.getSession();
        session.setAttribute("resetEmail", email);

        resp.sendRedirect("verify_otp.jsp?msg=Otp+sent");
    }
}

```

## ResendOtpServlet (explicit button to resend)

java

 Copy code

```

package com.yuvraj.servlet;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;
import java.sql.Timestamp;
import java.util.Random;

import com.yuvraj.config.SecurityConfig;
import com.yuvraj.dao.OtpDao;
import com.yuvraj.dao.UserDao;
import com.yuvraj.util.EmailUtil;

public class ResendOtpServlet extends HttpServlet {
    private OtpDao otpDao = new OtpDao();
    private UserDao userDao = new UserDao();

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String email = req.getParameter("email");
        if (userDao.getUserByEmail(email) == null) {
            resp.sendRedirect("forgot_password.jsp?msg=Email+not+found");
            return;
        }

        try {
            java.sql.ResultSet rs = otpDao.getOtpRow(email);

```

```

        boolean allowSend = true;
        if (rs != null && rs.next()) {
            Timestamp lastSent = rs.getTimestamp("last_sent_at");
            int sentCount = rs.getInt("sent_count");
            if (lastSent != null) {
                long millisSince = System.currentTimeMillis() - lastSent.getTime();
                long hourMillis = 60L * 60L * 1000L;
                if (millisSince <= hourMillis && sentCount >= SecurityConstants.SENT_COUNT_LIMIT) {
                    allowSend = false;
                } else if (millisSince > hourMillis) {
                    otpDao.resetSentCount(email);
                }
            }
        }

        if (!allowSend) {
            resp.sendRedirect("verify_otp.jsp?msg=Resend+limit+reached");
            return;
        }

        // generate and upsert new OTP
        int otp = new Random().nextInt(900000) + 100000;
        String otpStr = String.valueOf(otp);
        otpDao.upsertOtp(email, otpStr);

        String html = EmailUtil.otpTemplate(otpStr, email);
        EmailUtil.sendHtmlEmail(email, "Your Password Reset OTP - Resend");

        resp.sendRedirect("verify_otp.jsp?msg=Otp+resent");

    } catch (Exception e) { e.printStackTrace(); resp.sendRedirect("verify_otp.jsp?msg=Error"); }
}

```

## VerifyOtpServlet (with attempt counting and locking)

java

 Copy code

```

package com.yuvraj.servlet;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;

```

```

import java.sql.Timestamp;
import java.time.LocalDateTime;

import com.yuvraj.dao.OtpDao;
import com.yuvraj.dao.UserDao;
import com.yuvraj.config.SecurityConfig;

public class VerifyOtpServlet extends HttpServlet {
    private OtpDao otpDao = new OtpDao();
    private UserDao userDao = new UserDao();

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String email = (String) req.getSession().getAttribute("resetEmail");
        if (email == null) email = req.getParameter("email"); // fallback

        String enteredOtp = req.getParameter("otp");

        try {
            java.sql.ResultSet rs = otpDao.getOtpRow(email);
            if (rs == null || !rs.next()) {
                resp.sendRedirect("verify_otp.jsp?msg=No+OTP+found");
                return;
            }
            String otp = rs.getString("otp");
            Timestamp createdAt = rs.getTimestamp("created_at");
            int attempts = rs.getInt("attempts");

            // expiry check
            long ageMillis = System.currentTimeMillis() - createdAt.getTime();
            long expiryMillis = SecurityConfig.OTP_EXPIRY_MINUTES * 60L * 1000;
            if (ageMillis > expiryMillis) {
                resp.sendRedirect("verify_otp.jsp?msg=OTP+expired");
                return;
            }

            if (otp.equals(enteredOtp)) {
                // success
                otpDao.deleteOtp(email);
                // optional: store verified flag in session
                req.getSession().setAttribute("verifiedEmail", email);
            }
        }
    }
}

```

```

        resp.sendRedirect("reset_password.jsp?msg=verified");
    } else {
        // increment attempts
        int newAttempts = otpDao.incrementAttempts(email);
        if (newAttempts >= SecurityConfig.OTP_MAX_WRONG_ATTEMPTS) {
            // lock user account
            LocalDateTime until = LocalDateTime.now().plusMinutes(5);
            userDao.lockUser(email, Timestamp.valueOf(until));
            // delete otp record to require flow restart after lock
            otpDao.deleteOtp(email);
            resp.sendRedirect("verify_otp.jsp?msg=Account+locked+duration");
        } else {
            resp.sendRedirect("verify_otp.jsp?msg=Invalid+OTP+Attempts");
        }
    }
} catch (Exception e) {
    e.printStackTrace();
    resp.sendRedirect("verify_otp.jsp?msg=Error");
}
}
}

```

## ResetPasswordServlet (unchanged except extra checks)

Make sure you only allow reset if `verifiedEmail` exists in session (or allow email param and also check OTP table was cleaned). Example:

java

 Copy code

```

// in ResetPasswordServlet.doPost:
String verifiedEmail = (String) req.getSession().getAttribute("verifiedEmail");
if (verifiedEmail == null) {
    resp.sendRedirect("reset_password.jsp?msg=Not+authorized");
    return;
}
String newPass = req.getParameter("password");
String enc = PasswordEncrypt.hashPassword(newPass);
boolean f = dao.updatePassword(verifiedEmail, enc);
if (f) {
    // cleanup and redirect
    req.getSession().removeAttribute("verifiedEmail");
    resp.sendRedirect("login.jsp?msg=Password+Updated");
} else {

```

```
resp.sendRedirect("reset_password.jsp?msg=Failed");
}
```

---

## 6) Change Password (for logged-in users)

**JSP:** change\_password.jsp

jsp

 Copy code

```
<% com.yuvraj.model.User user = (com.yuvraj.model.User) session.getAttribute(
    if (user == null) { response.sendRedirect("login.jsp"); return; }
%>

<h2>Change Password</h2>
<form action="ChangePasswordServlet" method="post">
    Current Password: <input type="password" name="current" required><br>
    New Password: <input type="password" name="newpass" required><br>
    <button type="submit">Change</button>
</form>
```

**ChangePasswordServlet.java**

java

 Copy code

```
package com.yuvraj.servlet;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;

import com.yuvraj.dao.UserDao;
import com.yuvraj.model.User;
import com.yuvraj.security.PasswordEncrypt;

public class ChangePasswordServlet extends HttpServlet {
    private UserDao userDao = new UserDao();

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
```



```

HttpSession session = req.getSession();
User user = (User) session.getAttribute("userObj");
if (user == null) { resp.sendRedirect("login.jsp"); return; }

String current = req.getParameter("current");
String newpass = req.getParameter("newpass");

String encCurr = PasswordEncrypt.hashPassword(current);

// Verify current password
User check = userDao.login(user.getEmail(), encCurr);
if (check == null) {
    resp.sendRedirect("change_password.jsp?msg=Wrong+current+password");
    return;
}

// update password
String encNew = PasswordEncrypt.hashPassword(newpass);
boolean f = userDao.updatePassword(user.getEmail(), encNew);
if (f) {
    // optional: invalidate session to force re-login
    session.invalidate();
    resp.sendRedirect("login.jsp?msg=Password+changed.+Login+again");
} else {
    resp.sendRedirect("change_password.jsp?msg=Failed");
}
}
}

```

---

## 7) JSP UX additions

- On `verify_otp.jsp` add a **Resend OTP** button that posts to `ResendOtpServlet` with the email (or reads from session):

jsp

 Copy code

```

<form action="ResendOtpServlet" method="post">
    <input type="hidden" name="email" value="<%= session.getAttribute("resendOtpEmail") %>" />
    <button type="submit">Resend OTP</button>
</form>

```

- On `forgot_password.jsp` show messages from query params and a small help text about resend limits.
- 

## 8) web.xml mapping (examples)

Add servlet mappings:

xml

 Copy code

```
<servlet>
  <servlet-name>ForgotPasswordServlet</servlet-name>
  <servlet-class>com.yuvraj.servlet.ForgotPasswordServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>ForgotPasswordServlet</servlet-name>
  <url-pattern>/forgot</url-pattern>
</servlet-mapping>

<servlet>
  <servlet-name>ResendOtpServlet</servlet-name>
  <servlet-class>com.yuvraj.servlet.ResendOtpServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>ResendOtpServlet</servlet-name>
  <url-pattern>/resend-otp</url-pattern>
</servlet-mapping>

<servlet>
  <servlet-name>VerifyOtpServlet</servlet-name>
  <servlet-class>com.yuvraj.servlet.VerifyOtpServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>VerifyOtpServlet</servlet-name>
  <url-pattern>/verify-otp</url-pattern>
</servlet-mapping>

<servlet>
  <servlet-name>ChangePasswordServlet</servlet-name>
  <servlet-class>com.yuvraj.servlet.ChangePasswordServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>ChangePasswordServlet</servlet-name>
```

```
<url-pattern>/change-password</url-pattern>  
</servlet-mapping>
```

---

## 9) Security / Production notes (important)

1. **Use app-specific SMTP credentials** (Gmail App Password / dedicated Mail server).  
Never hardcode in source for production — use environment variables or encrypted config.
  2. **Consider using salted hashes** (e.g., bcrypt / Argon2) instead of bare SHA-256 for password storage. SHA-256 is OK if you also salt; bcrypt/Argon2 is preferred.
  3. **Rate-limit endpoints** at webserver or application level to avoid abuse (forgot password, resend).
  4. **Log suspicious activity** (many OTP requests / failed attempts).
  5. **Use HTTPS** for all forms and links (very important).
  6. **Invalidate OTP after use** (we delete row on success).
  7. **Consider storing a hashed OTP** rather than plaintext OTP in DB for extra safety (then compare hash of entered OTP).
  8. **CSRF protection** on forms (add CSRF tokens to forms used for password reset/changes).
-